

CHAPTER-THREE

Gathering User Requirements

❑ What is requirements gathering?

- **Requirements gathering** is the process of understanding what you are trying to build and why you are building it.
- **Requirements gathering** is often regarded as a part of developing software applications or of cyber-physical systems like aircraft, spacecraft, and automobiles (where specifications cover both software and hardware).
- It can, however, be applied to any product or project, from designing a new sailboat to building a patio deck to remodeling a bathroom.

Three Main Subprocesses of Requirements Gathering

- The key ingredients of the **requirements-gathering process** are three overlapping subprocesses: **requirements elicitation**, **requirements documentation**, and **requirements understanding**.
-

1. Requirements elicitation

- **Requirements elicitation** is the process of asking for and collecting top-level requirements from all relevant stakeholders.
- Efforts should be made to account for the needs of customers, their users, the internal stakeholders within your own company, and your key suppliers.
- **Requirements elicitation** focuses on describing the purpose of the system.
- The **client**, the **developers**, and the **users** identify a **problem area** and **define a system that addresses the problem**.

Three Main Subprocesses of Requirements Gathering

1. Requirements elicitation ...

- Such a definition is called **a requirements specification** and serves as a contract between the client and the developers.
- The **requirements specification** is **structured** and **formalized** during analysis to produce an analysis model.
- Both **requirements specification** and ***analysis model represent the same information.***
- They differ only in the **language** and **notation** they use; the **requirements specification** is written in **natural language**, whereas the **analysis model** is usually expressed in a **formal or semiformal notation**.

Three Main Subprocesses of Requirements Gathering

1. Requirements elicitation ...

- The **analysis model** supports **communication among developers**.
- They are both models of the system in the sense that they attempt to represent accurately the external aspects of the system.
- Given that both models represent the same aspects of the system, **requirements elicitation and analysis occur concurrently and iteratively**.
- Requirements elicitation and analysis focus **only** on *the user's view of the system*.

Three Main Subprocesses of Requirements Gathering

1. Requirements elicitation ...

- **For example**, the **system functionality**, the **interaction between the user and the system**, the **errors that the system can detect and handle**, and the **environmental conditions in which the system functions** are **part of the requirements**.
- The **system structure**, the **implementation technology selected to build the system**, the **system design**, the **development methodology**, and other aspects **not directly visible to the user** are **not part of the requirements**.

Three Main Subprocesses of Requirements Gathering

1. Requirements elicitation ...

- **Requirements elicitation includes the following activities:**
 - ***Identifying actors:*** During this activity, developers identify the different types of users the future system will support.
 - ***Identifying scenarios:*** During this activity, developers observe users and develop a set of detailed scenarios for typical functionality provided by the future system.
 - Scenarios are concrete examples of the future system in use.
 - Developers use these scenarios to communicate with the user and deepen their understanding of the application domain.

Three Main Subprocesses of Requirements Gathering

1. Requirements elicitation ...

- **Requirements elicitation includes the following activities:**

- **Identifying use cases:** Once developers and users agree on a set of scenarios, developers derive from the scenarios a set of use cases that completely represent the future system.

- Whereas scenarios are concrete examples illustrating a single case, use cases are abstractions describing all possible cases.
- When describing use cases, developers determine the scope of the system.

- **Refining use cases:** During this activity, ***developers ensure that the requirements specification*** is complete by detailing each use case and describing the behavior of the system in the presence of errors and exceptional conditions.

Three Main Subprocesses of Requirements Gathering

1. Requirements elicitation ...

- **Requirements elicitation includes the following activities:**
 - ***Identifying relationships among use cases:*** During this activity, developers identify dependencies among use cases.

 - They also consolidate the use case model by factoring out common functionality.
 - This ensures that the requirements specification is consistent.
 - ***Identifying nonfunctional requirements:*** During this activity, developers, users, and clients agree on aspects that are visible to the user, but not directly related to functionality.
 - These include constraints on the performance of the system, its documentation, the resources it consumes, its security, and its quality.

Three Main Subprocesses of Requirements Gathering

1. Requirements elicitation ...

- During **requirements elicitation**, developers access many different sources of information, including **client-supplied documents** about the application domain, manuals and **technical documentation of legacy systems** that the future system will replace, and most importantly, the **users and clients themselves**.
- Developers interact the most with users and clients during requirements elicitation.

Three Main Subprocesses of Requirements Gathering

1. Requirements elicitation ...

- We focus on *two methods for eliciting information*, making decisions with users and clients, and managing dependencies among requirements and other artifacts:
 - **Joint Application Design (JAD)** focuses on building consensus **among developers, users, and clients** by jointly developing the requirements specification.
 - **Traceability** focuses on **recording**, *structuring*, *linking*, **grouping**, and **maintaining** dependencies among requirements and between requirements and other work products.

Three Main Subprocesses of Requirements Gathering

Requirements Elicitation Concepts

- **Functional Requirements (FR)**
- **Nonfunctional Requirements (NFR)**

■ **Functional Requirements (FR)**

- **Functional requirements** describe the interactions between the system and its environment independent of its implementation.
- The environment includes the user and any other external system with which the system interacts.

Three Main Subprocesses of Requirements Gathering

Requirements Elicitation Concepts ...

■ **Nonfunctional Requirements**

- *Nonfunctional requirements* describe aspects of the system that are not directly related to the functional behavior of the system.
- *Nonfunctional* requirements include a broad variety of requirements that apply to many different aspects of the system, from usability to performance.
- The **FURPS+** model² used by the Unified Process [Jacobson et al., 1999] provides the following categories of nonfunctional requirements:
 - ✓ **Usability** is the ease with which a user can learn to operate, prepare inputs for, and interpret outputs of a system or component.
 - ✓ Usability requirements include, for example, conventions adopted by the user interface, the scope of online help, and the level of user documentation.
 - ✓ Often, clients address usability issues by requiring the developer to follow user interface guidelines on color schemes, logos, and fonts.

Three Main Subprocesses of Requirements Gathering

Requirements Elicitation Concepts ...

■ **Nonfunctional Requirements ...**

- **Reliability** is the ability of a system or component to perform its required functions under stated conditions for a specified period of time.

- **Reliability requirements** include, for example, an acceptable mean time to failure and the ability to detect specified faults or to withstand specified security attacks.
- More recently, this category has often been replaced by dependability, which is the property of a computer system such that reliance can justifiably be placed on the service it delivers.
- **Dependability** includes **reliability**, **robustness** (the degree to which a system or component can function correctly in the presence of **invalid** inputs or stressful environment conditions), and safety (a measure of the absence of catastrophic consequences to the environment).

Three Main Subprocesses of Requirements Gathering ...

2. Requirements documentation organizes the input from the requirements elicitation process into whatever format is appropriate for your organization. This formatting may include:

- ✓ User stories
- ✓ Functional decompositions (especially for complex cyber-physical systems)
- ✓ Feature descriptions

Three Main Subprocesses of Requirements Gathering ...

- These will be collected in a **top-level requirements** specification like a **product requirements document (PRD)** or a system specification.
 - The purpose of this top-level specification is to make those stories and descriptions available to all members of the project team.
- 3. Requirements confirmation** is the process of making sure all stakeholders and team members have a common understanding of what you're trying to build.
- This involves reviewing and refining the requirements.
 - It will very likely require additional elicitation and revision of the documentation as well.

Fundamental requirements gathering techniques

- Individually interview people informed about the operation and issues of the current system and future systems needs.
- **In reality there are hundreds of different techniques for requirement elicitation.**
- some commonly used techniques are mentioned:
 - ✓ **Interview**
 - ✓ **Questionnaires**
 - ✓ **Brainstorming**
 - ✓ **Storyboard**
 - ✓ **Prototyping**
 - ✓ **Use cases**
 - ✓ **JAD (Joint Application Development)**

Fundamental requirements gathering techniques ...

1). Interview

- **Interview** is common technique used for gather data or information.
- In this technique **interviewer** conduct a meeting with **interviewee**.
- Interviews questions should be according to the interviewee's level.
- Gather information according to his/her requirement.
- Questions should be open ended however; interviewee can provide clear answer of your question.

Fundamental requirements gathering techniques ...

- **There are three types of questions.**
- These are **structured**, **unstructured** and **semi structured**.
- **Structured interviews** are conducted where **domain is specified**.
- In this type specific questions are asked and get to the point answer.
- In this way, all the questions are covered up in this type.
- The other is **unstructured** interview in this type interviewer ask questions and require detail answer of these questions.

Fundamental requirements gathering techniques ...

- Interviewer applies only partial control over the way of discussion.
- In this way some topic may be neglected.
- **Semi structured** is combination of both.
- The **semi structured interview**, where elementary usual of the question is organized and used.

2). Questionnaires

- **Questionnaire** is best technique for gathering information.
- In this technique questions are listed in paper.
- Questions are filled by the **stakeholders** and **get the answer of these questions.**

Fundamental requirements gathering techniques ...

3). Brainstorming

- **Brainstorming** technique is group discussion in which members shares their ideas and find out the solution of specific problem.
- Brainstorming generates or gathers **new ideas rather** than **its quality**.
- This technique is more popular because of it is a group activity all the members share own idea.
- It is more productive for **the reason that groups**.
- When members generate idea it is more value able as of group
- product and members enjoy the group activity.

Fundamental requirements gathering techniques ...

- There is method for conducting brainstorming task. These are :
 - ✓ **Subjects and design**
 - ✓ **Procedure**
 - ✓ **Results**
 - ✓ **Discussion**
- It is not compulsory to manage brainstorming sessions for resolve major issues.
- The purpose of this technique is introductory mission statement for specific problem.
- Advantage of this technique is encouraging open-minded and free ideas or innovation on particular predefine problem.

Fundamental requirements gathering techniques ...

3). Storyboard

- In this technique **user**, **customer** and **developers** draw picture of what they want to develop software.
- Draw picture of all requirement like **tool bar**, **main window**, **dialogue boxes** etc.
- After draw full picture of all requirements, all members agreed upon on it.
- It is just like **paper prototyping**.
- **Storyboarding** is very common technique for designing about which you want to get information for their project.

Fundamental requirements gathering techniques ...

- **Storyboarding** is much realistic for understanding about software's structure for unknown persons who do not know about technical terms.
- There are some attribute or elements of storyboarding, which explain basic points for draw storyboarding.

These are:-

- ✓ **Level of details**
- ✓ **Inclusion of text**
- ✓ **Inclusion of people and emotions**
- ✓ **Number of frames**
- ✓ **Portrayal of time**

Fundamental requirements gathering techniques ...

- **Study business documents/document analysis** to discover reported **issues**, **policies**, **rules**, and **directions** as well as concrete examples of the **use of data** and **information in the org.**
- What can the analysis of documents tell you about the requirements for a new system?
- In documents you can find information about the following :
 - ✓ **Problems with existing systems (e.g., missing information or redundant steps)**
 - ✓ **Opportunities to meet new needs if only certain information or information processing were available (e.g., analysis of sales based on customer type)**

Fundamental requirements gathering techniques ...

- ✓ **Organizational direction that can influence information system requirements**
(e.g., trying to link **customers** and **suppliers** more closely to the organization)
- ✓ **Titles and names of key individuals who have an interest in relevant existing systems** (e.g., the name of a sales manager who led a study of the buying behavior of key customers)
- ✓ **Values of the organization or individuals who can help determine priorities for different capabilities desired by different users** (e.g., maintaining market share even if it means lower short-term profits)

Essential Use Case Modeling

- To **model a system**, the most important aspect is to capture the **dynamic behavior**.
- **Dynamic behavior** means the behavior of the system when it is **running/operating**.
- **Only static behavior is not sufficient to model a system** rather dynamic behavior is more important than static behavior.
- In UML, there are **five** diagrams available to model the dynamic nature and use case diagram is one of them.
- Now as we have to discuss that the use case diagram is dynamic in nature, there should be some internal or external factors for making the interaction.

Essential Use Case Modeling ...

- These **internal** and **external** agents are known as **actors**.
- Use case diagrams consists of **actors**, **use cases** and **their relationships**.
- The diagram is used to model the **system/subsystem of an application**.
- A single use case diagram captures a particular functionality of a system.
- Hence to model the entire system, a number of use case diagrams are used.

Essential Use Case Modeling ...

- Support **requirements engineering activities** and **the requirement process**
- **Capture what a system is supposed to do**, i.e., systems functional requirements
- Describe sequences of actions a system performs that yield an observable result of value to a particular actor.
- Model actions of the system at its external interface
- Capture how the system coordinates human actions

Essential Use Case Modeling ...

- **Use cases** provide a **high level view of the system**.
- They capture to a certain extent system structures.
- Use case describe sequences of actions a system performs that yield an observable result of value to a particular actor.

□ **Sequence of actions:-**

- ✓ set of functions, **algorithmic procedures**, **internal processes**, etc.
- ✓ **System performs:** system functionalities
- ✓ An observable result of value to a user
- ✓ A particular actor: individual or device

Essential Use Case Modeling ...

Purpose of Use Case Diagrams

- The purpose of **use case diagram** is to capture the **dynamic aspect of a system**.
- However, this definition is too generic to describe the purpose, as other **four diagrams** (**activity**, **sequence**, **collaboration**, and **Statechart**) also have the same purpose.
- We will look into some specific purpose, which will distinguish it from other **four** diagrams.
- **Use case diagrams** are used to gather the requirements of a system including internal and external influences.

Essential Use Case Modeling ...

- These requirements are mostly **design requirements**.
- Hence, when a system is analyzed to gather its **functionalities**, **use cases are prepared** and **actors are identified**.
- When the initial task is complete, use case diagrams are modelled to present the outside view.
- In brief, the purposes of use case diagrams can be said to be as follows:-
 - ✓ **Used to gather the requirements of a system.**
 - ✓ **Used to get an outside view of a system.**
 - ✓ **Identify the external and internal factors influencing the system.**
 - ✓ **Show the interaction among the requirements and actors.**

Essential Use Case Modeling ...

- ✓ Relatively easy to write and easy to read
- ✓ Comprehensible by users
- ✓ Engage the users in the requirements process
- ✓ Force developers to think through the design of a system from a user viewpoint
- ✓ Identify a context for the requirements of the system
- ✓ Critical tool in the **design**, **implementation**, **analysis** and **testing** process
- ✓ **Rapid change allows exploratory approach**
- ✓ Serve as inputs to the user documentation

Essential Use Case Modeling ...

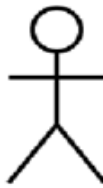
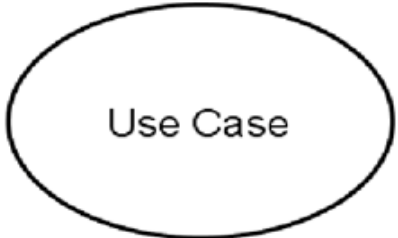
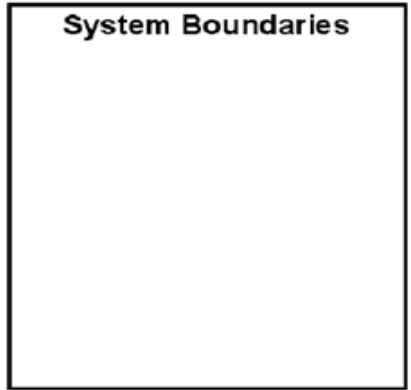
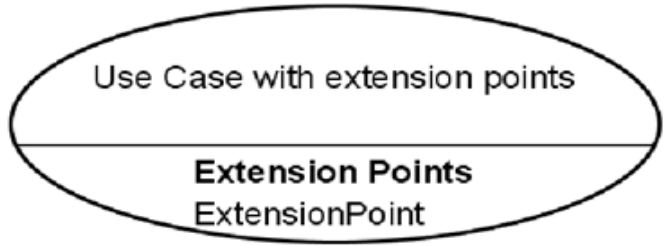
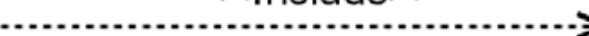
- **Use Cases modelling** is an effective means of communicating with users and other stakeholders about the system and what is intended to do.
- Use Cases support a relationship with **scenarios** and **relevant activities** (e.g., testing)

Use Cases: Strengths and Weaknesses

- **Strengths**
 - Capture different actors views of the system
 - Capture some structures in requirements
 - Are comprehensible by naïve users
- **Weaknesses**
 - Lack of non-functional requirements
 - Lack of what the system shall not do

Use Cases at a Glance

Use Cases at a Glance

 Actor	 Use Case	Generalization 
 System Boundaries	 Use Case with extension points Extension Points ExtensionPoint	<<Extend>> 
		<<Include>> 

Use Cases at a Glance ...

□ Anatomy of Use Cases: Basic Diagrams

- ✓ **Actors** are represented as stick figures
- ✓ **Use Cases** as ellipses
- ✓ **Lines** represent **associations** between these things
- ✓ **Use Case diagrams** show who is involved with what

Use Cases at a Glance ...

❑ Use Cases Basics:-

- ✓ **Actors:** An Actor is external to a system, interacts with the system, may be a human user or another system, and has a goals and responsibilities to satisfy in interacting with the system.
- ✓ **Use Cases:** identify functional requirements, which are described as a sequence of steps describe actions performed by a system capture interactions between the system and actors.
- ✓ **Relationships:** Actors are connected to the use cases with which they interact by a line which represents a relationship between the actors and the use cases.
- ✓ **System Boundaries:** Identify an implicit separation between actors (external to the system) and use cases (internal to the system)

Actors and Use Cases

■ Finding nonhuman actors

- ✓ Incorporating other systems (e.g., databases)
 - ✓ Ignoring internal components
-
- ✓ Input/Output Devices

■ Roles of the Actors

■ Naming the Actors

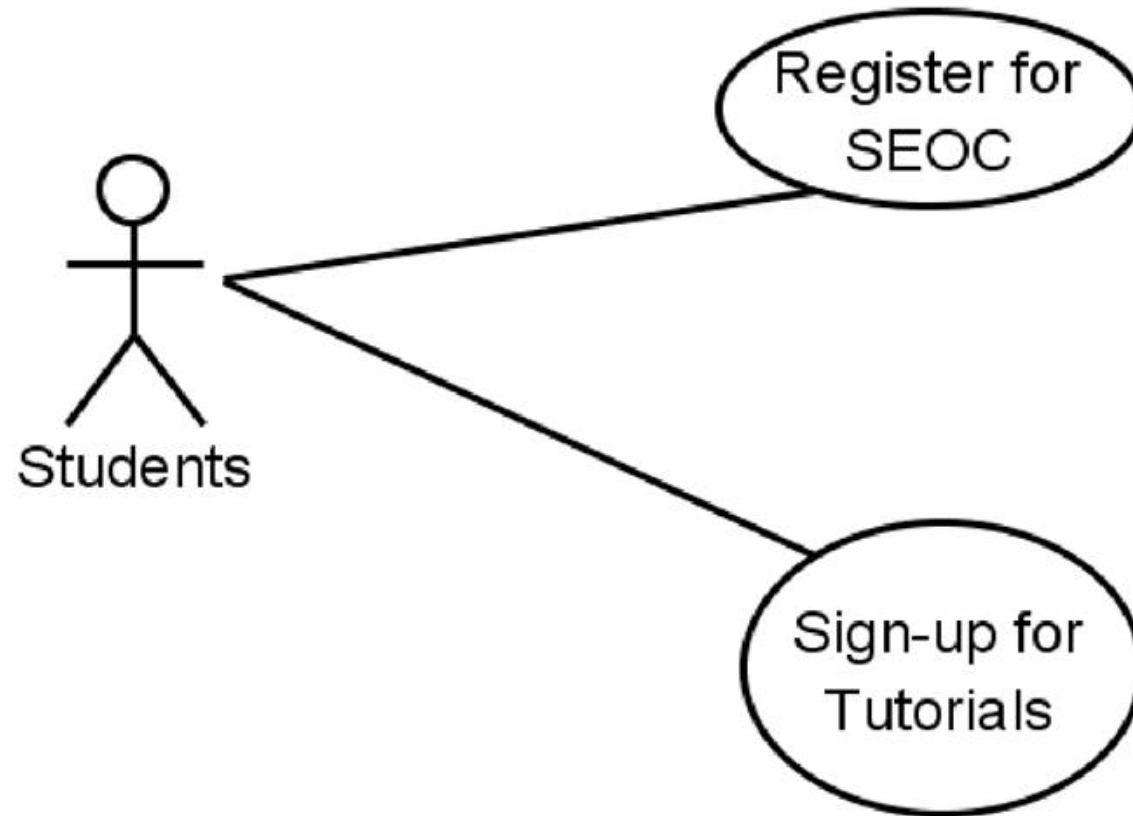
Actors and Use Cases ...

- **Here are some general hints:**

- ✓ Take care to identify generic actors who do a particular task, or cover a particular role with respect to the system
- ✓ Do not get confused with job titles
- ✓ Use case diagrams should not be too complex
- ✓ Aim for reasonably generic use cases
- ✓ Try not be too detailed at first.

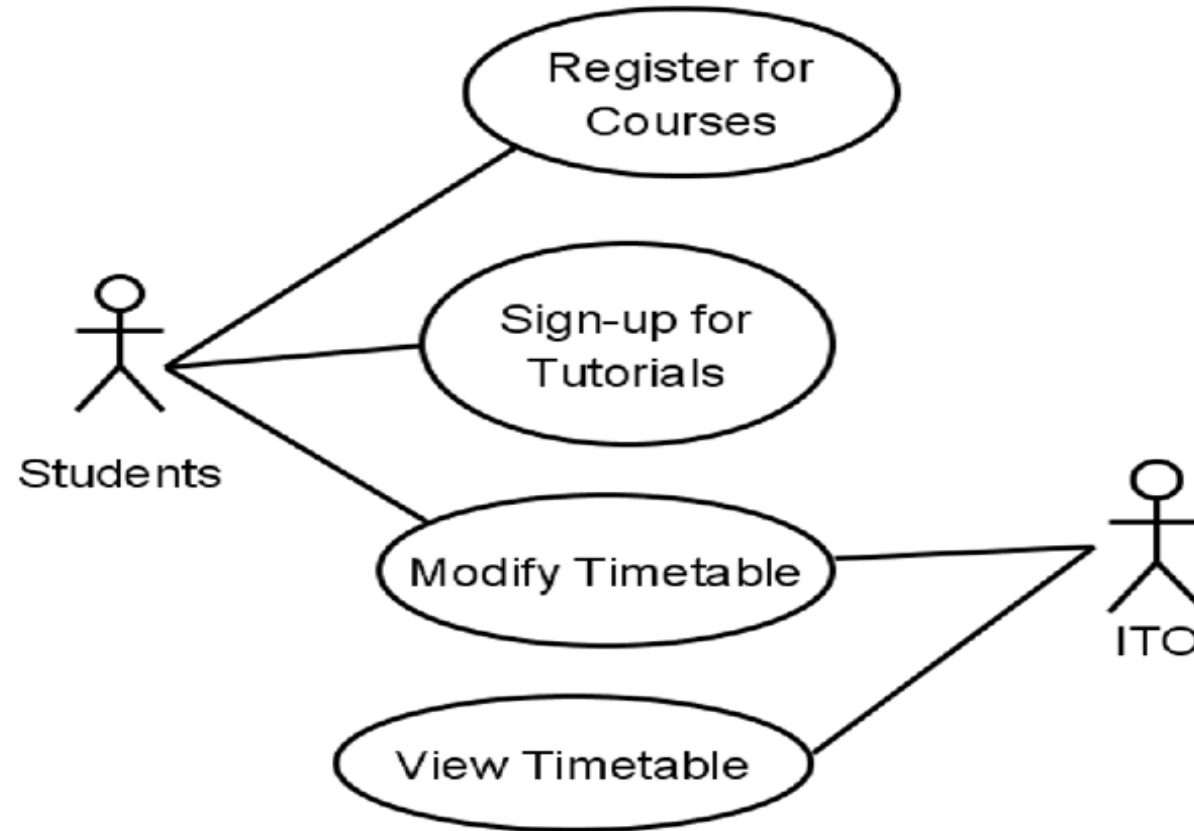
Actors and Use Cases ...

Actors and Use Cases



Actors and Use Cases ...

Actors and Use Cases



Actors and Use Cases ...

Finding Actors

The actors can be identified by answering a number of questions:

- ✓ Who will use the main functionality of the system?
- ✓ Who will need support from the system to do their daily tasks?
- ✓ Who will need to maintain and administer the system, and keep it working?
- ✓ Which hardware devices does the system need to handle?
- ✓ With which other systems does the system need to interact?
- ✓ Who or what has an interest in the results (the value) that system produces?

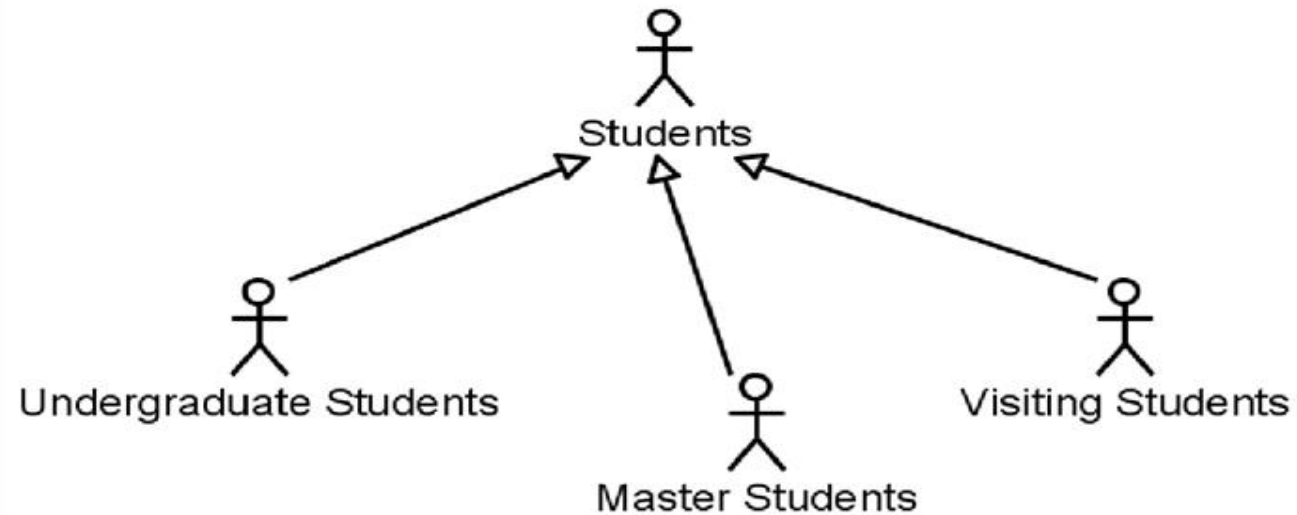
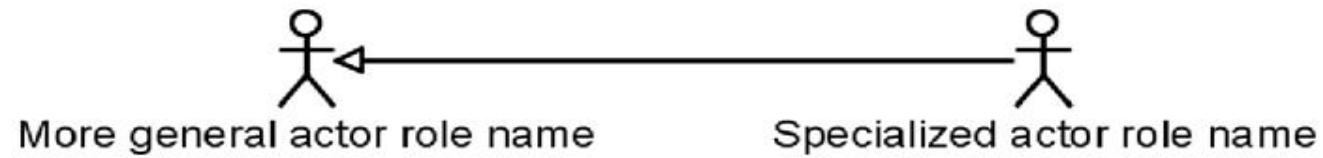
Actors and Use Cases ...

Finding Actors ...

- Despite the simplicity of use cases, it is difficult to identify the involved actors and use cases.
- One of the common issue is the completeness of the involved actors and relevant use cases.
- This is often due to a lack of understanding of the system and its requirements.
- Hence, use cases help to discuss an high-level structured view of the system, its functionality and the relevant actors around the system.
- Another common difficulty is the identification of the trade-offs between generality and specificity.

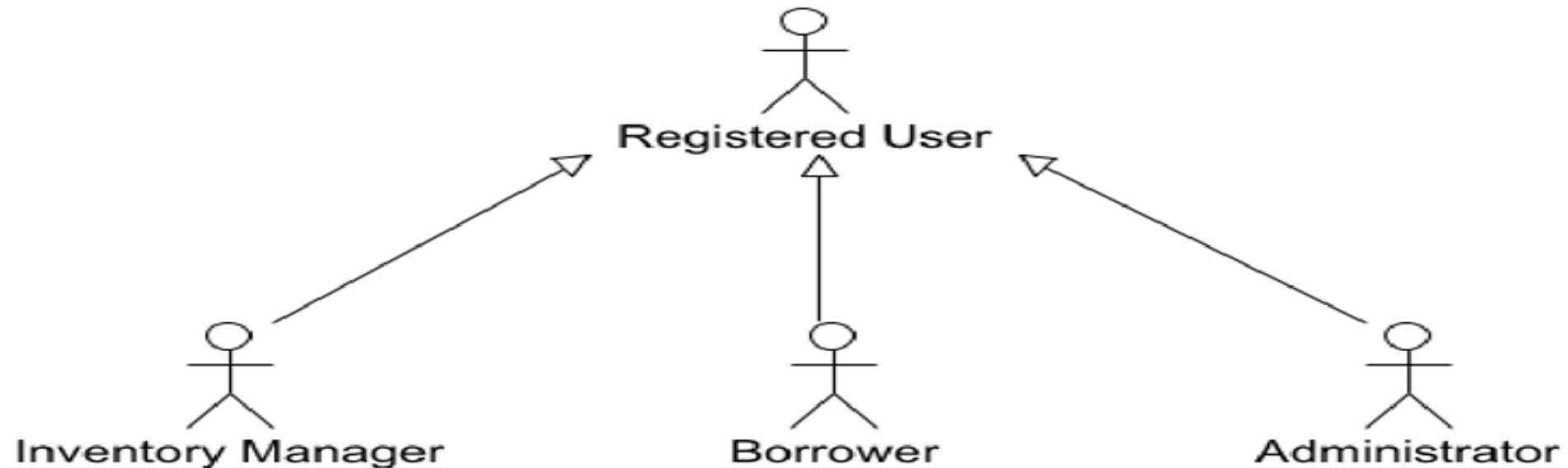
Generalizations between Actors

Generalizations between Actors



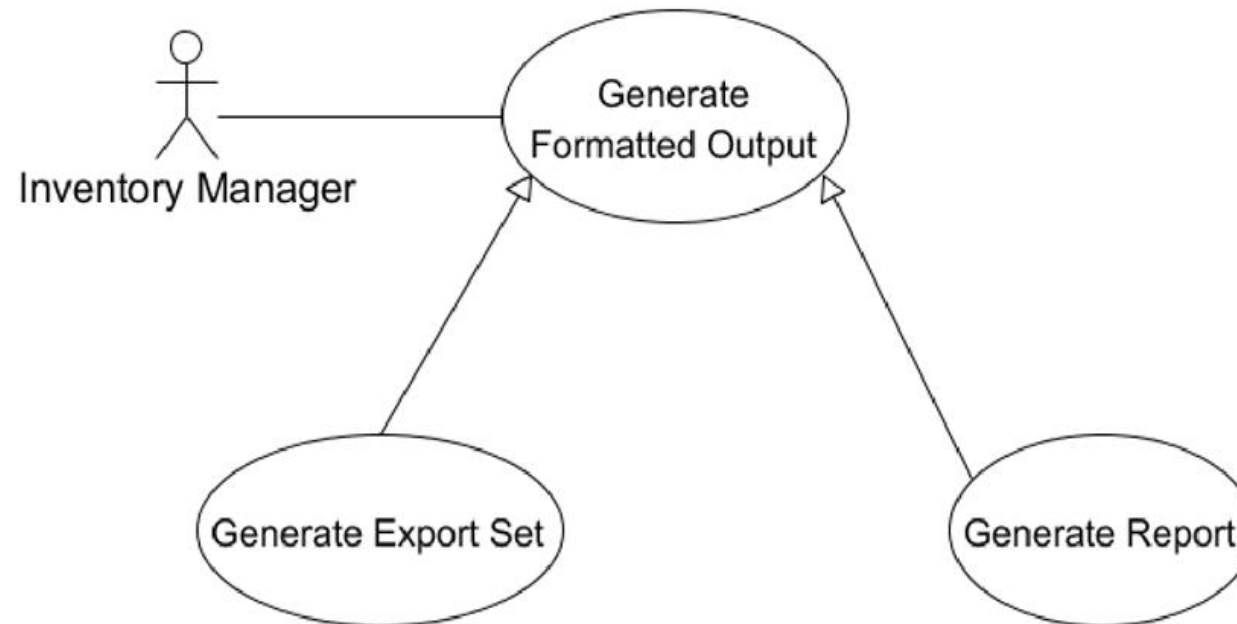
Generalizations between Actors ...

- Actors may be similar in how they use the system (e.g., project and system managers)
- An Actor generalization indicates that instances of the more specific actor may be substituted for instances of the more general actor



Generalizations between Actors ...

- Indicate that the more specific use case receives or inherits the actors, behaviour sequences, and extension points of the more general use case
- ***Generalization is often implemented by inheritance.***



Finding Use Cases

- **For each identified actor, ask the following questions:**
 - *Which functions does the actor require from the system? What does the actor need to do?*

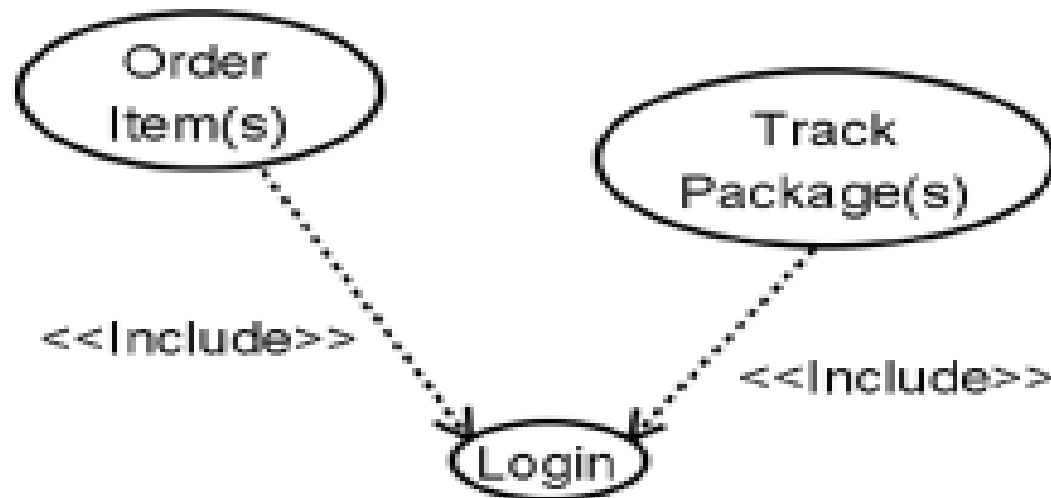
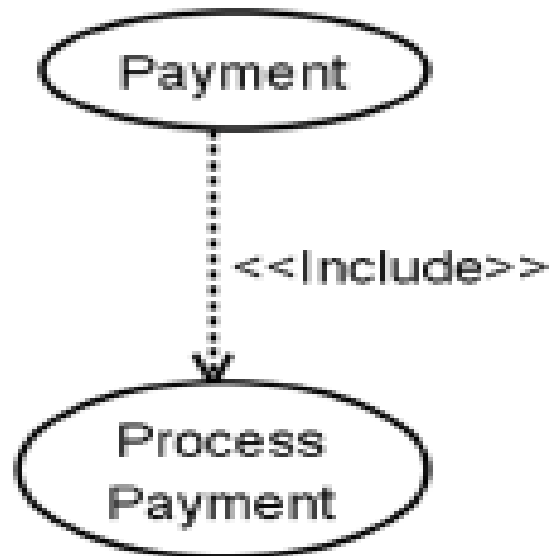
 - *Does the actor need to read, create, destroy, modify, or store some kind of information in the system?*
 - *Does the actor have to be notified about events in the system, or does the actor need to notify the system about something?*
 - *What do those events represent in terms of functionality?*

Finding Use Cases ...

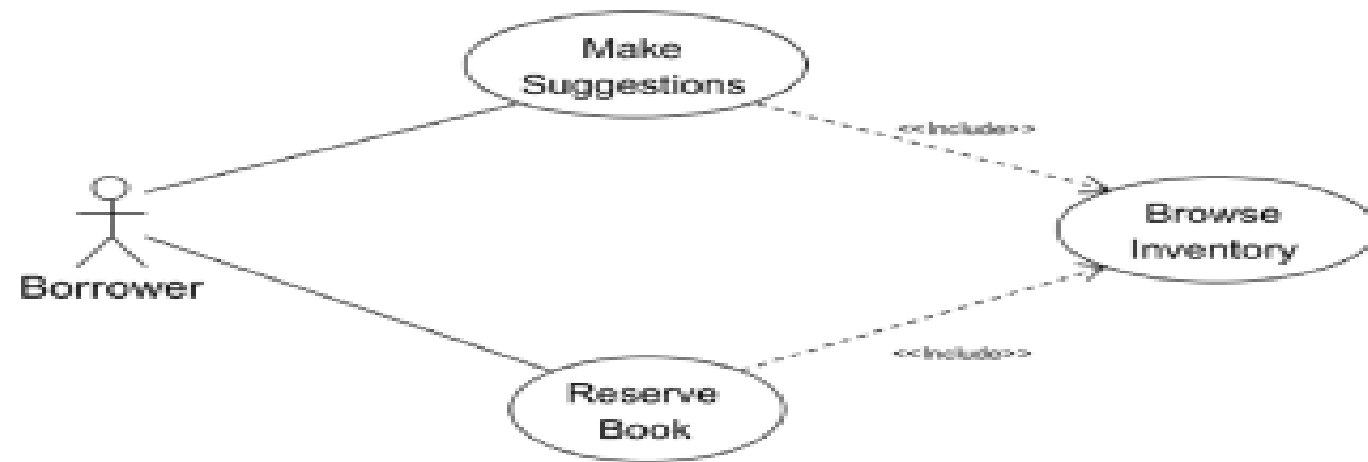
- The <<**include**>> relationship **holds** when one use case is included in others
- The <<**include**>> relationship **declares** that a use case reuses another one being included
- The ***included use case*** is (typically not complete on its own) a required part of other use cases
- An include relationship shows how common behaviour in a number of use cases
- can be described in a shared use case that is included in the other use cases

<<include>> Relationship

<<include>> Relationship



<<include>> Relationship ...

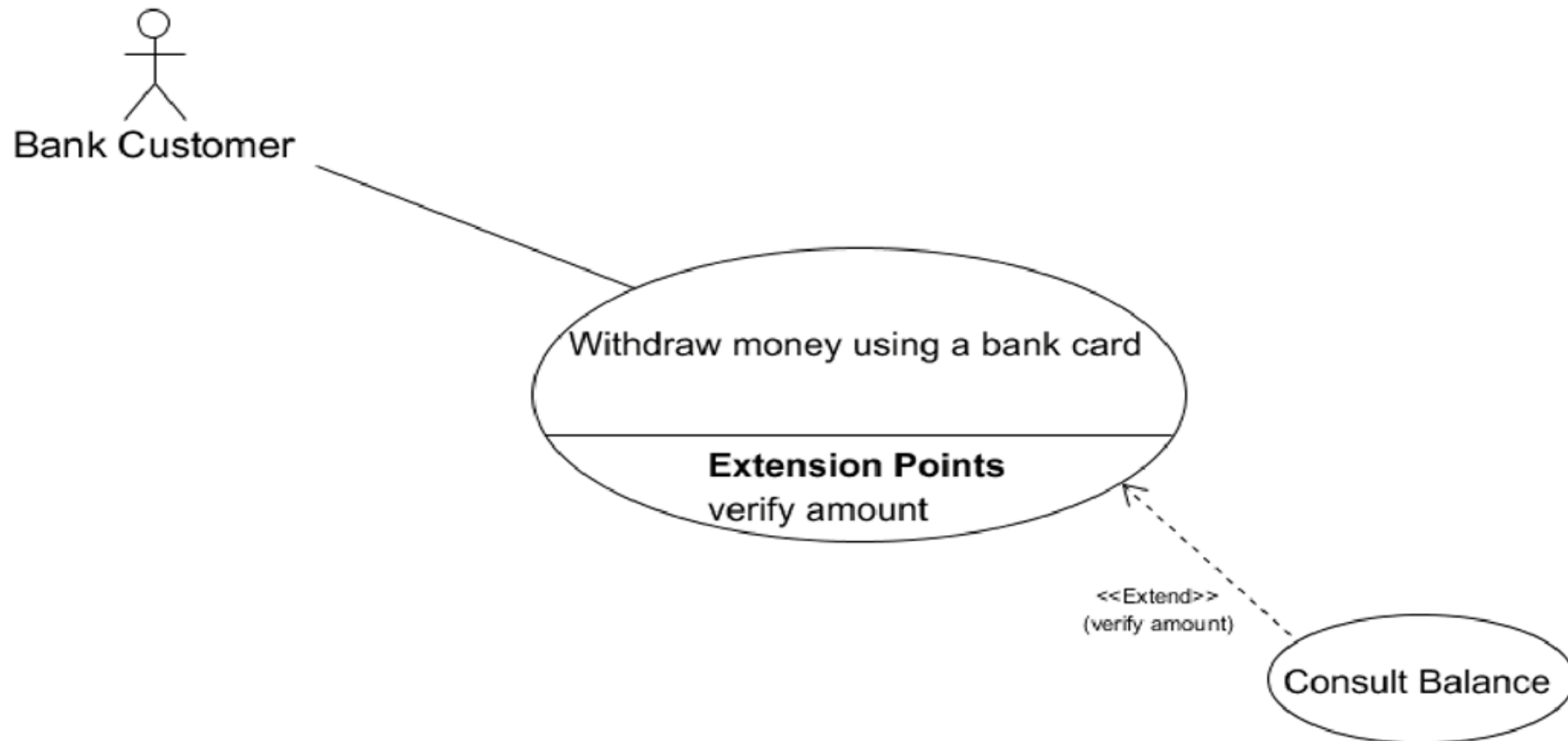


<<extend>> Relationship

- The **<<extend>> relationship** holds when use cases extend, i.e., optionally provide other functionalities to extended use cases
- A use case may be extended by (i.e., completely reuse) another use case, but this is optional and depends on runtime conditions or implementation decisions
- Any use case you want to extend must have clearly defined extensions points

<<extend>> Relationship ...

<<extend>> Relationship



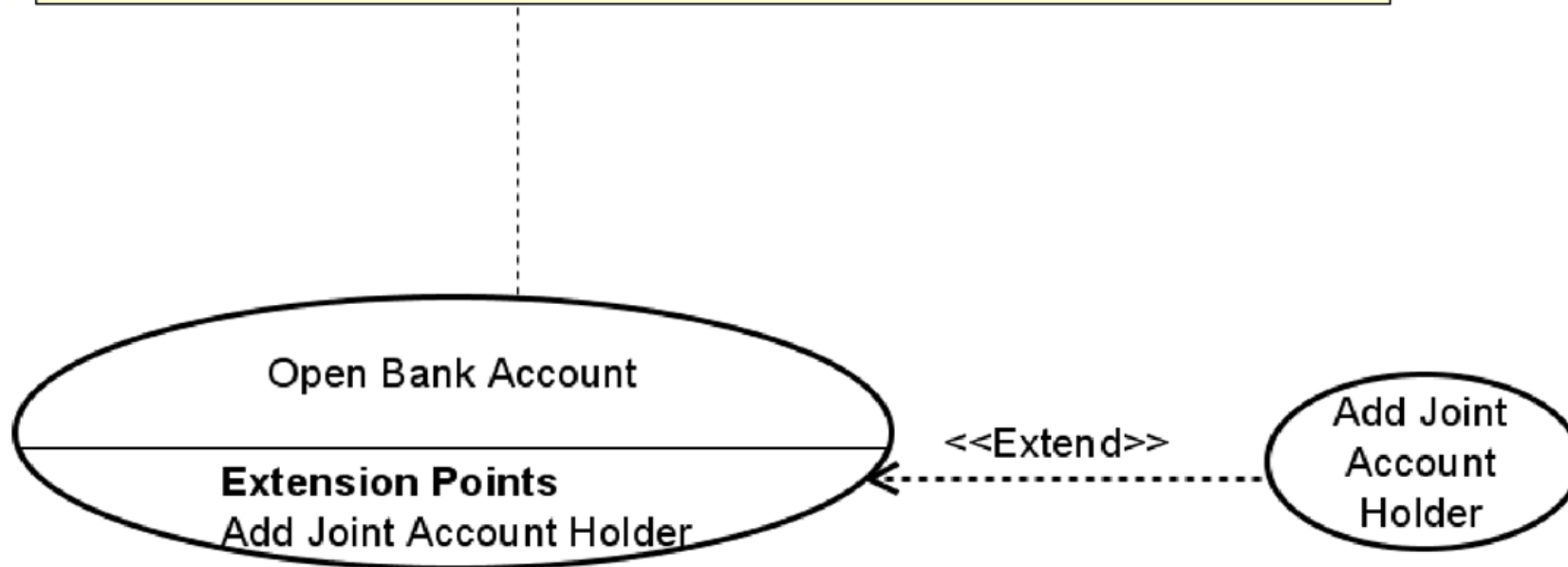
<<extend>> Relationship ...

"Add Joint Account Holder" extends "Open Bank Account"

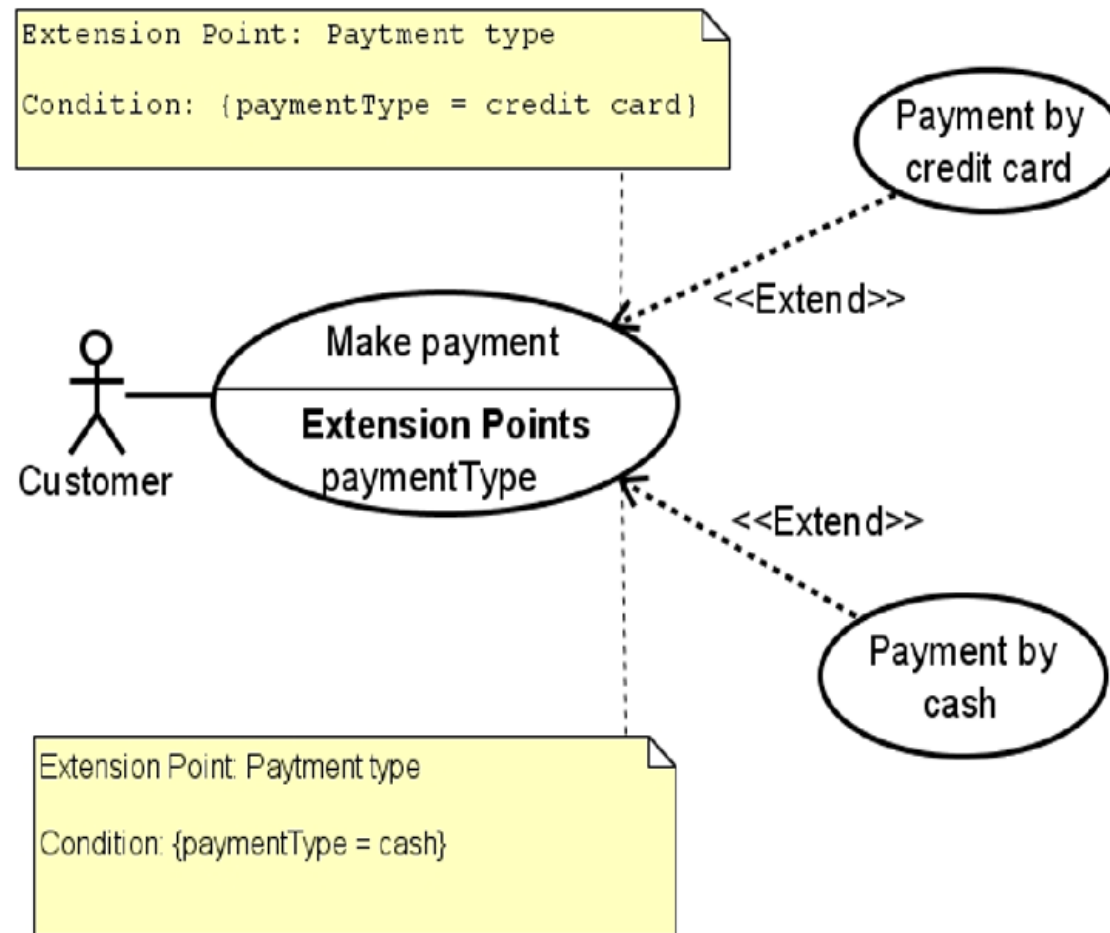
"Open Bank Account" is a valid use case on its own

Condition: {Number of Account Holder(s) > 1}

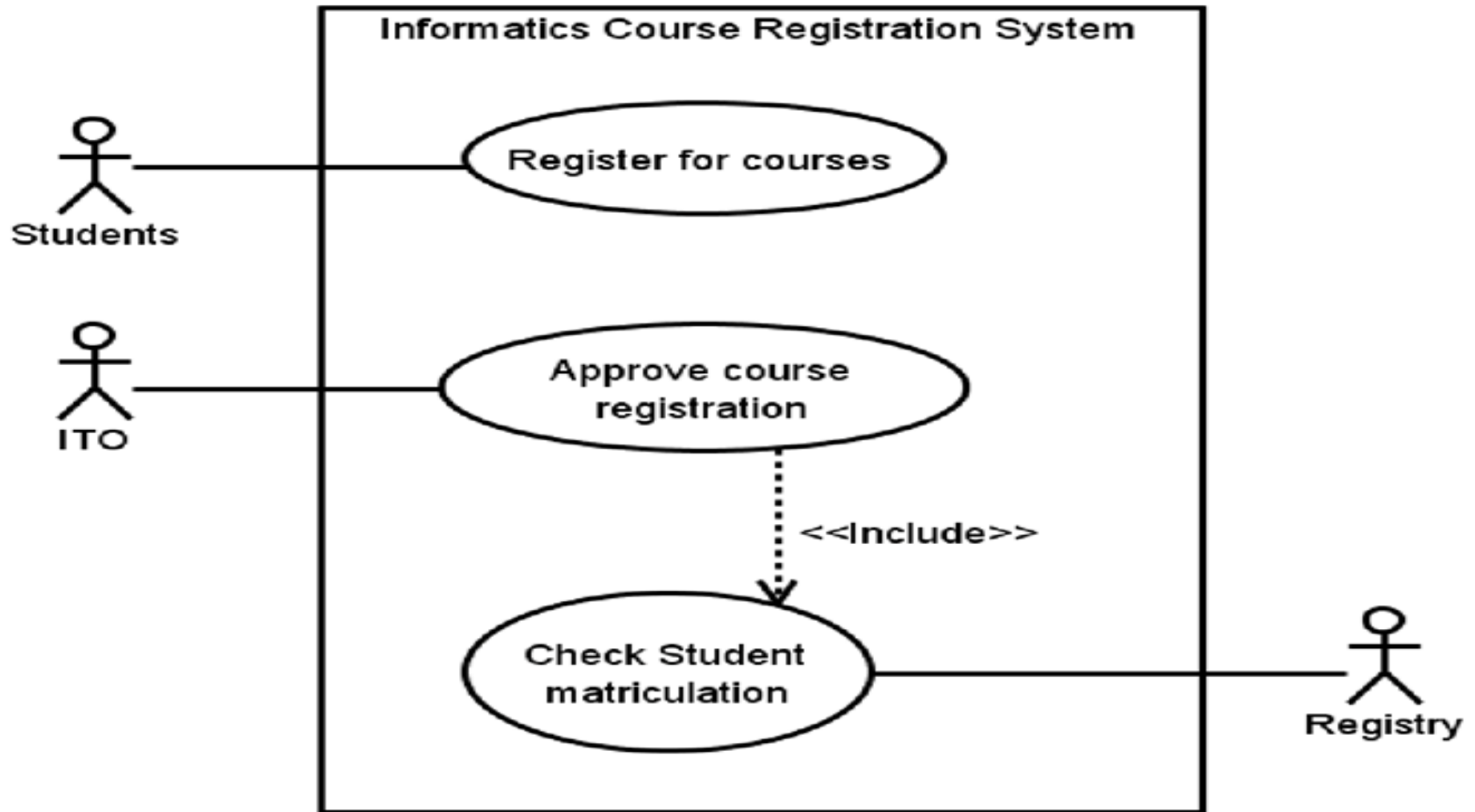
Extension Point: Add Joint Account Holder



<<extend>> Relationship ...



System Boundaries



System Boundaries ...

- Identify an implicit separation between **actors** (**external to the system**) and **use cases** (**internal to the system**)
- The system boundaries identify what is part of the system and the actors interacting with it.
- The boundaries affect the functionalities that the system has to implement/support.
- Therefore, there are both technical (whether the system needs to implement a specific functionality or the functionality is provided by an external actor) as well as business implications (e.g., financial).

System Boundaries ...

- **Note that** it is possible to specify ***multiplicities*** between **actors** and **use cases**.
- It is useful to capture various information (e.g., concurrency) already in the use cases.

- However, it is useful initially to maintain the use case diagrams as general as possible in order to avoid (or commit) to particular design during early stages of the requirements process.

Use Case Descriptions

- A **use case description** complements each **use case** in the diagram
 - Identify use case information
-

Warnings: avoid to specify design information

- A use case main course (of actions) is a generic sequence of actions undertaken in using the system
 - Identify **pre** and **post** conditions
 - **Identify alternate courses**
- Provide generic test scenarios for the full system
- Templates capture/structure use case information

Use Case Descriptions ...

- Description Example : **Use Case name: Register for Courses**

Description: This use cases allows students to register for informatics courses.

- The student uses the Informatics Course Registration System, an online system, for selecting the courses to attend for the forthcoming semester.

- **Main course:**

1. This use case starts when a student visits the system web page

1.1 The system provides the list of available courses in the forthcoming semester

2. The student identifies the courses and select them

3. The student confirm the selection, which is then recorded

■

Use Case Descriptions ...

- Description Example 2 : **Use Case name: request an appointment with a GP (General Practitioner).**
- **Description:** **An system allows patients to request appointments with GPs.**

- **Main course:**
 1. A patient requests appointment to the system
 2. The system queries a scheduler for available GPs and times
 3. The system responds with GPs and times
 4. The system negotiates with Patient on suitable GP/time
 5. The system confirms GP/time with the Scheduler
 6. The scheduler responds with confirmation of appointment (e.g., booking number)
 7. The system communicates confirmation to Patient

A Basic Use Case Template

A Basic Use Case Template

Use Case [number #]	<i>the name is the goal as a short active verb phrase</i>	
Goal in Context	<i>a longer statement of the goal, if needed</i>	
Preconditions	<i>what we expect is already the state of the world</i>	
Success End Condition	<i>the state of the world upon successful completion</i>	
Failed End Condition	<i>the state of the world if goal abandoned</i>	
Primary Actor	<i>a role name or description for the primary actor</i>	
Secondary Actors	<i>other systems relied upon to accomplish use case</i>	
Trigger	<i>the action upon the system that starts the use case</i>	
Description	Step	Action
	1	
	...	
Extensions or Variations	Step	Branching Action
	...	<i>condition causing branching action or name of sub-use case</i>

How to Create Use Cases

Step 1. Identify and Describe the **Actors**

Step 2. Identify and Describe the **Use Cases**

Step 3. Identify the (**Actor** and **Use Case**) Relationships

Step 4. Individually **Outline** Use Cases

Step 5. **Prioritize** the Use Cases

Step 6. **Refine** the Use Cases

How to Create Use Cases ...

Simple questions or checklist to support the specification of use cases.

Step 1. Identify and Describe the **Actors**: **who** uses the system? who manages the system? who maintains the system? **Who** provides information to the system? Who gets information from the system? etc.

Step 2. Identify and Describes the **Use Cases**: What will the actor use the system for? Will the actor create, store, change, remove or read information in the system? etc.

Step 3. Identify the **Actor** and the **Use Case** Relationships

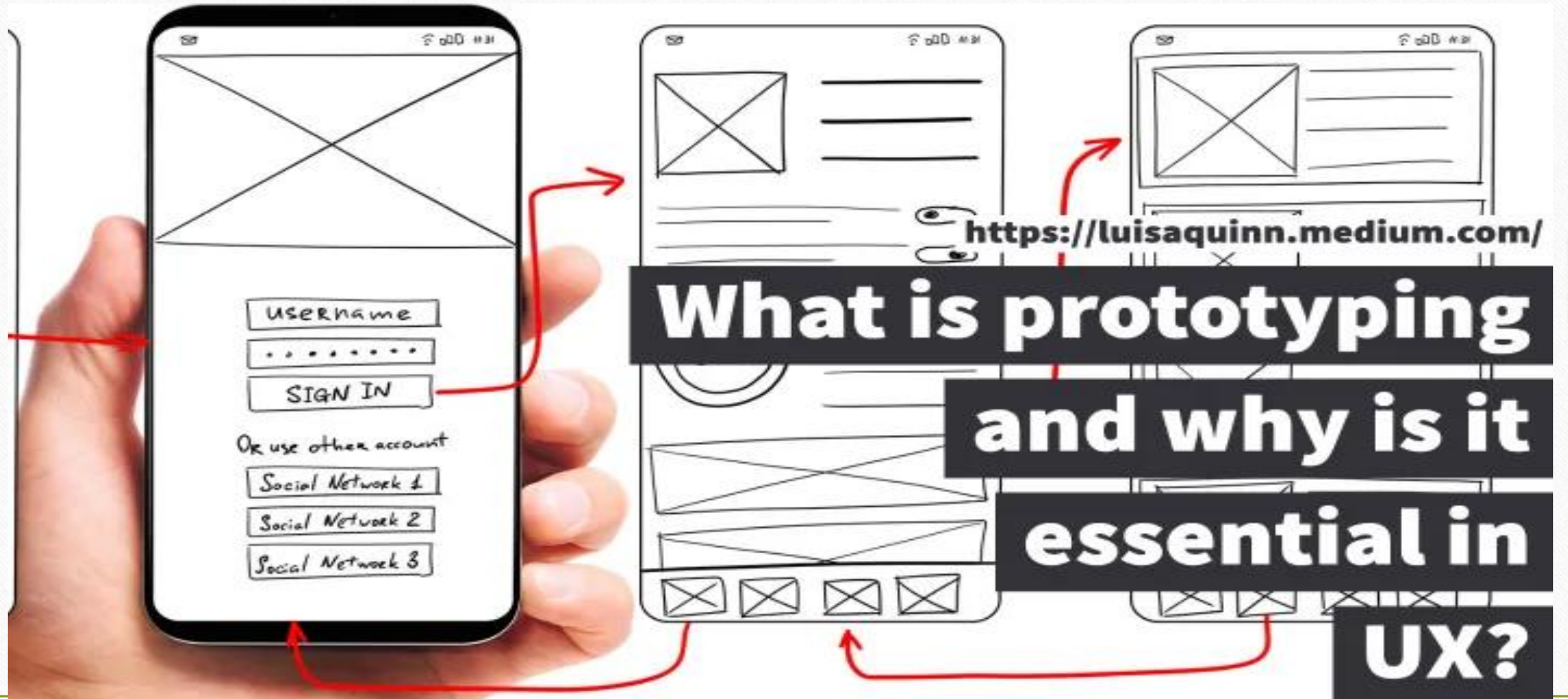
Step 4. **Outline** the individual Use Cases

How to Create Use Cases ...

Step 5. **Prioritize** the use cases: for instance, on the basis of utility or frequency of use depending on the process this may be closely linked to what is needed in the process

Step 6. **Refine** the Use Cases: Develop each use case (starting with the priority ones) develop the associated use case structure the use case

Essential User Interface Prototyping



Essential User Interface Prototyping

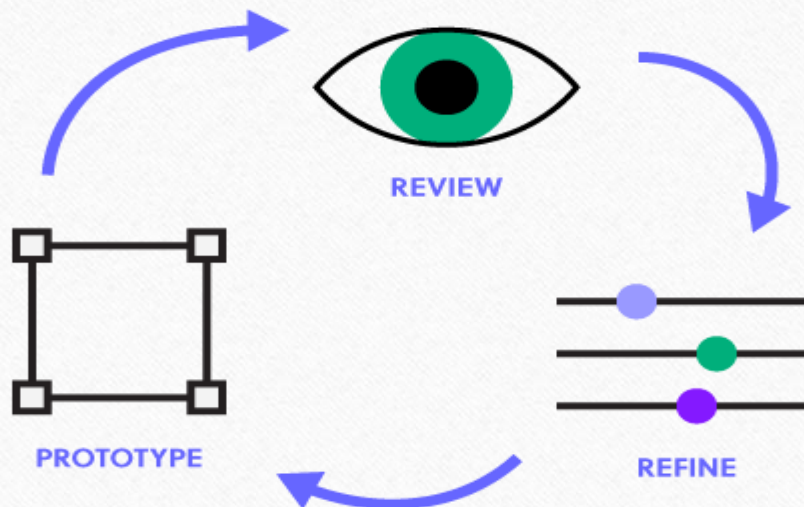
- ❖ A **prototype** is an initial version of a system which may be used for experimentation
- ❖ Prototypes are valuable for requirements elicitation because users can experiment with the system and point out its strengths and weaknesses.
- ❖ Rapid development of prototypes is essential so that they are available early in the elicitation process.
- ❖ A **prototype** is a simulation of a final product which product teams use for testing before committing resources to building the actual thing.
- The **goal of a prototype** is **to test and validate** ideas before sharing them with **stakeholders** and eventually passing the final designs **to engineering teams** for the development process.

Essential User Interface Prototyping ...

- ❖ **Prototypes** are essential for **identifying** and **solving** **user pain points** with participants during usability testing.
- ❖ **Testing prototypes** with end-users enables UX teams to visualize and optimize the user experience during the design process.
- ❖ **Engineering** is expensive, and making changes to a final product is often not as straightforward as teams anticipate.
- ❖ So, finding and fixing errors during the design process is critical.
- ❖ Prototypes have **four main qualities**:
 - ❖ **Representation** :The prototype itself, i.e., paper and mobile, or HTML and desktop.
 - ❖ **Precision** :The fidelity of the prototype, meaning its level of detail—low-fidelity or high-fidelity.
 - ❖ **Interactivity** : The functionality available to the user during the testing phase, e.g., fully functional, partially functional, or view-only.
 - ❖ **Evolution** : The lifecycle of the prototype. Some are built quickly, tested, thrown away, and then replaced with an improved version (known as “rapid prototyping”). Others may be created and improved upon, ultimately evolving into the final product.

Essential User Interface Prototyping ...

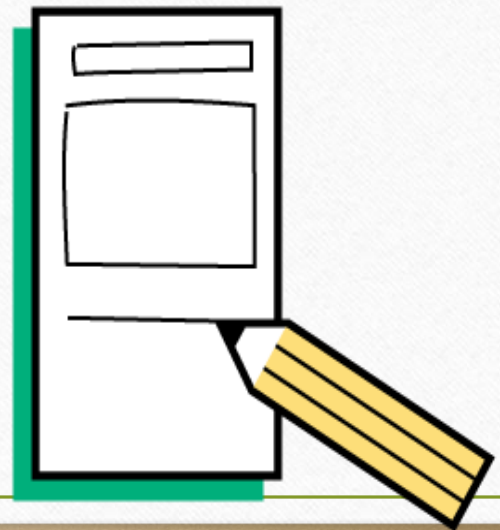
- ❖ Another **common misconception** about prototyping is that it only needs to be done once or twice at the end of the design process—**not true**.
- ❖ One of our mottos that we believe at **UXPin** is “**test early and test often**.”
- ❖ According to Elementor’s Director of UX, the website building platform’s designers’ – average four to five prototyping sessions, depending on the complexity of a given design.
- ❖ You **should prototype every possible iteration of your design** even your early basic ideas for solving a user need.
- ❖ Prototyping shouldn’t be reserved only for beta tests of the final version; you should test any and every version of your product!



Approaches to prototyping

❖ Paper prototyping

- a paper mock-up of the system is developed and used for system experiments.
- **A paper prototype** is a prototype that is drawn on a paper or a digital whiteboard.
- Such a prototype is used during the **early design stages**, like a design thinking workshop while designers still brainstorm ideas.
- It works best during early design stages where design teams collaborate to explore many concepts fast.
- Team members sketch ideas by hand using simple lines, shapes, and text.
- The emphasis is on lots of ideas and speed, not aesthetics.



Approaches to prototyping ...

- UX Teams lay paper screens on the floor, table, or pinned to a board to simulate user flows. A common practice for testing these prototypes is to have one person play “the product,” switching the sketches according to how the real user behaves.

A low visual/low functional paper prototype.



Approaches to prototyping ...

■ Advantages of Paper Prototypes:

- ✓ **Fast** : You can sketch a prototype in minutes, which is why paper works so well for testing lots of ideas..
- ✓ **Inexpensive** : You only need a marker pen and paper to create prototypes, making the process cheap and accessible.
- ✓ **Team-building** : Paper prototyping is a collaborative effort, and often teams have fun coming up with fresh ideas.
- ✓ **Documentation** : Team members can keep physical copies of paper prototypes, notes, and to do so for quick reference during future iterations.

■ Disadvantages of Paper Prototypes:

- ✓ **Unrealistic** — No matter how skilled the art or craftsmanship, paper prototypes will never be more than hand-drawn representations of a digital product.
- ✓ **False positives** — Sometimes, paper prototypes don't validate ideas properly.
- ✓ What seems like a good idea on paper might not work effectively in a digital wireframe.
- ✓ **No gut reactions** — Paper prototypes rely on the user's imagination, adding a break between seeing the stimulus and responding to it. That "gut" reaction is crucial for a successful UX.

Approaches to prototyping

❖ 'Wizard of Oz' prototyping

- a person simulates the responses of the system in response to some user inputs.

❖ Executable prototyping

- a fourth generation language or other rapid development environment is used to develop an executable prototype.

Types of prototyping

❑ **Throw-away prototyping**

- ✓ intended to help elicit and develop the system requirements.
- ✓ The requirements that should be prototyped are those that cause most difficulties to customers and which are the hardest to understand. Requirements that are well-understood need not be implemented by the prototype.

❑ **Evolutionary prototyping**

- ✓ intended to deliver a workable system quickly to the customer.
- ✓ Therefore, the requirements that should be supported by the initial versions of this prototype are those which are well-understood and which can deliver useful end-user functionality.
- ✓ It is only after extensive use that poorly understood requirements should be implemented.

Prototyping Benefits

- The prototype allows users to experiment and discover what they really need to support their work.
- Establishes feasibility and usefulness before high development costs are incurred.
- Essential for developing the 'look and feel' of a user interface.
- Can be used for system testing and the development of documentation.
- Forces a detailed study of the requirements which reveals inconsistencies and omissions.

Prototyping costs and problems

- **Training costs** - prototype development may require the use of special purpose tools.
- **Development costs** - depend on the type of prototype being developed.
- Extended development schedules developing a prototype may extend the schedule although the prototyping time may be recovered because rework is avoided.
- **Incompleteness** - it may not be possible to prototype critical system requirements.

Domain modeling with class responsibility collaborator (CRC) cards

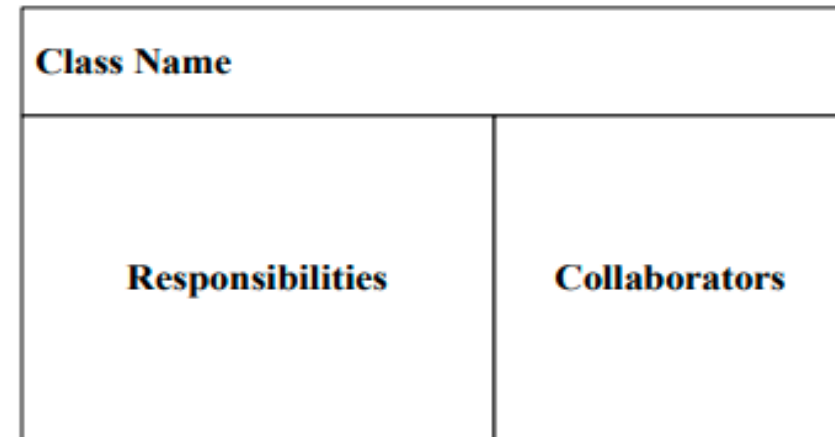
- **Class-Responsibility-Collaboration (CRC)** cards are a brainstorming tool used in the design of object-oriented software.
- They were originally proposed by **Ward Cunningham** and **Kent Beck** as a teaching tool, but are also popular among expert designers and recommended by extreme programming supporters.
- Members of a brainstorming session will write up one CRC card for each relevant class/object of their design.

Domain modeling with class responsibility collaborator (CRC) cards

■ CRC card components

A CRC card is partitioned into **three** areas:

- ✓ On top of the card, the **class name**
- ✓ On the left, the **responsibilities** of the class
- ✓ On the right, **collaborators** (other classes) with which this class interacts to fulfill its responsibilities



CRC card layout.

Domain modeling with class responsibility collaborator (CRC) cards ...

- A class represents a collection of similar objects.
- An object is a person, place, thing, event, concept, screen, or report that is relevant to the system at hand.
- For example, Figure in next slide shows a shipping/inventory control system with classes such as **Inventory Item**, **Order**, **Order Item**, **Customer**, and **Surface Address**.
- The name of the class appears across the top of the card.
- A responsibility is anything that a class knows or does. For example, customers have names, customer numbers, and phone numbers.
- These are the things that a customer knows. Customers also order products, cancel orders, and make payments.
- These are the things that a customer does.

Domain modeling with class responsibility collaborator (CRC) cards ...

■ CRC Models

- A **CRC model** is a collection of **CRC cards** that represent the whole or part of an application or problem domain.
- The most common use for CRC models, the one that this white paper addresses, is to gather and define the user requirements for an object-oriented application.
- . Figure 2 presents an example CRC model for a shipping/inventory control system, showing the CRC cards as they would be placed on a desk or work table.
- Note the placement of the cards: Cards that collaborate with one another are close to each other, cards that don't collaborate are not near each other.

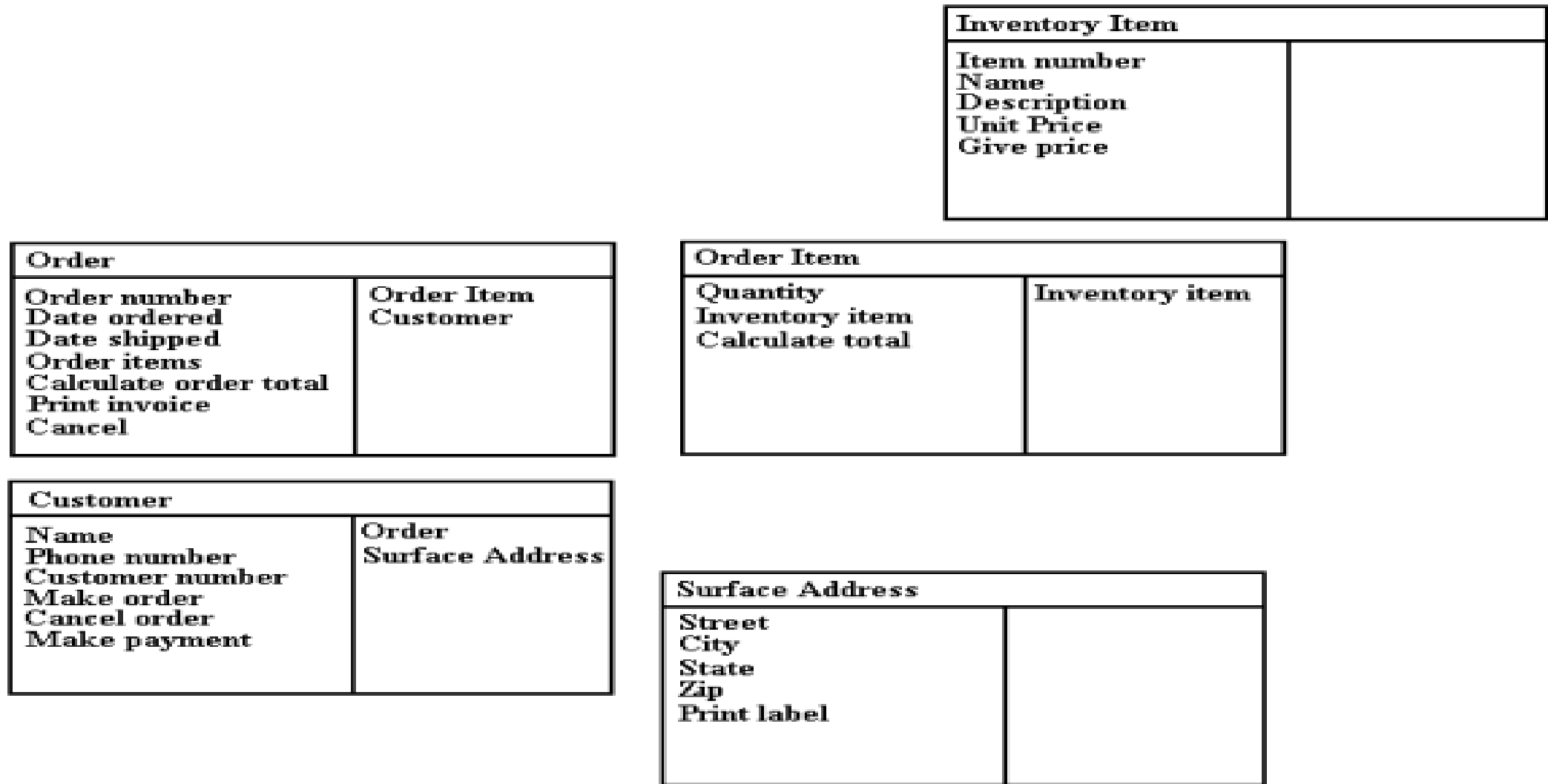


Figure 2. A CRC model for a simple shipping/inventory control system.

Domain modeling with class responsibility collaborator (CRC) cards

- **The steps of CRC modeling are:**

- ❖ Find classes

- ❖ Find responsibilities

- ❖ Define collaborators

- ❖ Define use-cases

- ❖ Arrange the cards on the table

Domain modeling with class responsibility collaborator (CRC) cards ...

■ Finding Classes:

- ✓ Look for anything that interacts with the system, or is part of the system
 - ✓ Ask yourself “Is there a customer?”
-
- ✓ Follow the money
 - ✓ Look for reports generated by the system
 - ✓ Look for any screens used in the system
 - ✓ Immediately prototype interface and report classes
 - ✓ Look for the three to five main classes right away
 - ✓ Create a new card for a class immediately
 - ✓ Use one or two words to describe the class
 - ✓ Class names are singular

Domain modeling with class responsibility collaborator (CRC) cards ...

■ **Finding Responsibilities :**

- ✓ Ask yourself what the class knows
 - ✓ Ask yourself what the class does
-
- ✓ If you've identified a responsibility, ask yourself what class it "belongs" to
 - ✓ Sometimes get responsibilities that we won't implement, and that's OK
 - ✓ Classes will collaborate to fulfil many of their responsibilities

■ **Defining Collaborators :**

- ✓ Collaboration occurs when a class needs information that it doesn't have
- ✓ Collaboration occurs when a class needs to modify information that it doesn't have
- ✓ There will always be at least one initiator of any given collaboration
- ✓ Sometimes the collaborator does the bulk of the work
- ✓ Don't pass the buck
- ✓ New responsibilities may be created to fulfil the collaboration

Domain modeling with class responsibility collaborator (CRC) cards ...

■ **Defining Use-Cases :**

- ✓ The business domain experts (BDEs) will identify them as responsibilities of actor classes
- ✓ Do some brainstorming
- ✓ Transcribe the scenarios onto cards

■ **Moving the Cards Around :**

- ✓ Cards that collaborate with each other should be close to one another on the desk
- ✓ The more that two cards collaborate, the closer that they should be on the desk
- ✓ Expect to be moving the cards around a lot at the beginning
- ✓ Put "busy" cards towards the center of the table
- ✓ Actually move them around
- ✓ People will identify relationships/associations between classes as they move them around

Domain modeling with class responsibility collaborator (CRC) cards ...

- **How CRC Modeling Fits In**
- In general the definition of **use cases** and **prototypes** generally come **before a CRC model**, which in turn generally **comes before a class diagram**.
- **The reason for this is simple:** Use cases and **user interface prototypes** are **less detailed than CRC models**, which in turn are less detailed than class diagrams.
- **Yes**, while you are CRC modeling you will draw sketches, or prototypes of screens.
- **Yes**, while you are CRC modeling you will identify new use cases and/or update existing ones.
- In general, however, you will find that it is better to start out “simple” with use cases and prototypes, get a little more complicated with CRC cards, and then get even more complicated with class diagrams.

Domain modeling with class responsibility collaborator (CRC) cards ...

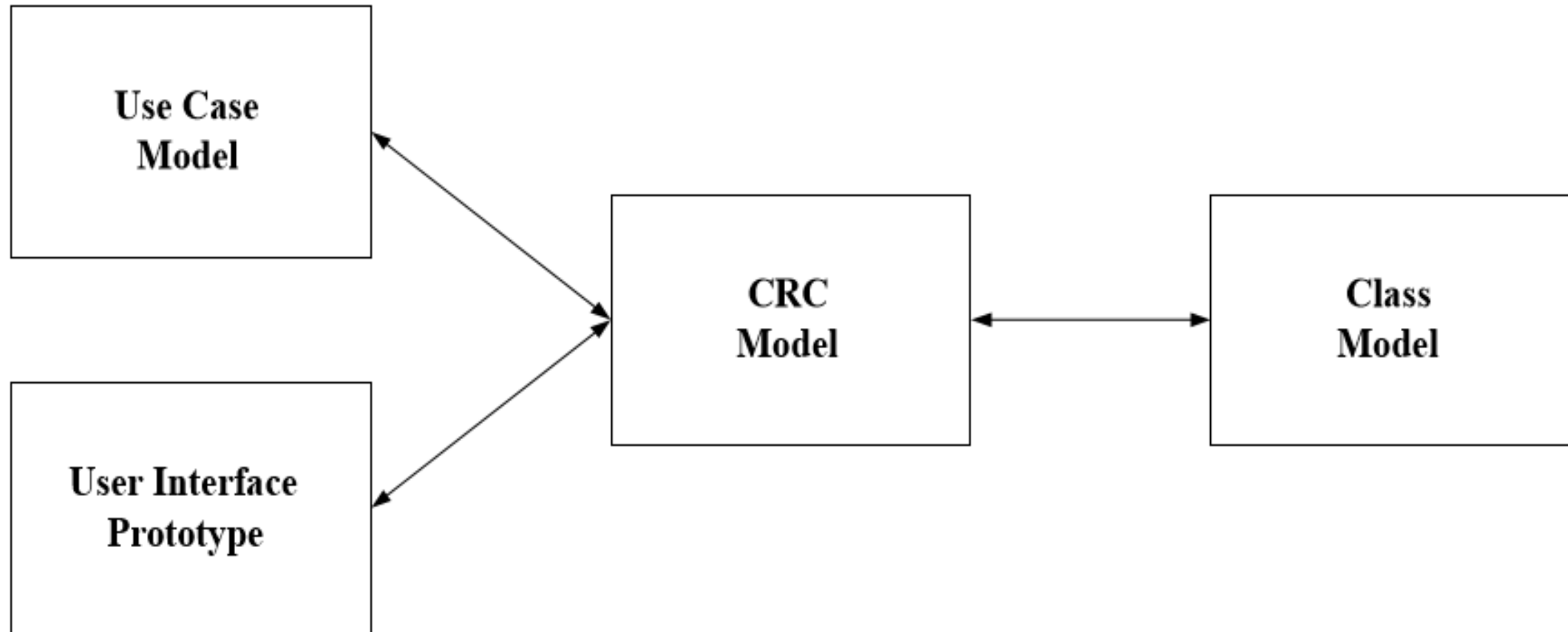


Figure 4. How CRC modeling fits in.

Domain modeling with class responsibility collaborator (CRC) cards ...

■ **The Advantages of CRC Modeling:**

There are many advantages to CRC modeling (Ambler, 1995), which include:

- 1. The experts do the analysis:** The people who understand the problem domain, the business domain experts (BDEs) are the people who create the model.
- 2. User participation increased:** Because users are actively involved in defining the model their satisfaction with the work will be much greater. CRC modeling increases user buy-in to the project.
- 3. Breaks down communication barriers :** Users and developers work together side-by-side to create the CRC model.
- 4. It's simple and straightforward :** You get a group of people together in a room and fill out a bunch of index cards. Because of its simplicity you can explain CRC modeling to a group of people in 10 or 15 minutes.

Domain modeling with class responsibility collaborator (CRC) cards ...

■ The Advantages of CRC Modeling ...

5. It's non-threatening to users: Many people are afraid, and rightly so, of losing their jobs to automation. Few people, if any, are afraid of losing their jobs to a stack of index cards.

6. It's inexpensive and portable : You can buy 100 index cards for two or three dollars and throw them into your briefcase.

7. It goes hand-in-hand with prototyping : CRC modeling and prototyping are both iterative processes in which users are greatly involved. It is very common to draw rough sketches of screens and reports during CRC modeling sessions.

8. It leads directly into class diagramming : CRC models and class diagrams show many of the same concepts. In many ways class diagrams are simply supersets of CRC models.

Domain modeling with class responsibility collaborator (CRC) cards ...

- **The Disadvantages of CRC Modeling:**

- There are several, albeit minor, **disadvantages** to **CRC modeling**:

1. It's threatening to some developers : Too many developers do not recognize the need to work closely with users, thinking that because they know the technology they also understand the business domain.

2. It's hard to get users together : Scheduling people for meetings is always hard to do. Try to keep the number of people in the room to just a few key people and try to plan at least a week in advance.

3. CRC cards are limited : CRC models are just part of the definition of user requirements for an OO application, you should also consider use cases, prototypes, and formal requirements documents.

END OF CHAPTER - THREE !